

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Tutorial #01 C++ Expressions, Statements, Arrays & Pointers



Overview



- **First example program**
- **Reading data from standard input**
- **Writing data to standard output**
- **Compiling & executing program**
- **Use of streams**
- **Use of namespace**



First example program



Example - 01



- Reading int from standard input
- A C++ program that uses printf and scanf statements
- stdio.h is part of standard C library
- It provides functions for performing input and output operations.
- printf, scanf, putchar, getchar

```
#include<iostream>
#include<stdio.h>

int main(int argc, char *argv[])
{
    int v1;
    scanf("%d", &v1);
    Printf("%d\n", v1);
}
```

```
Compiling
$ gcc hello.c -g
```

```
Executing
$ ./a.out
```

Example - 02



- Formatting specification rests on user
- To read int use %d in scanf/printf
- To read char use %c in scanf/printf
- To read float use %f in scanf/printf

```
#include<iostream>
#include<stdio.h>

int main(int argc, char *argv[])
{
    int v1; char c1; float f1;
    scanf("%d", &v1);
    scanf("%c", &c1);
    scanf("%f", &f1);

}
```



Example - 03

- Errors cannot be handled by these functions
- Number of format specifiers must match number of arguments
- Use of scanf and printf are rigid

```
#include<iostream>
#include<stdio.h>

int main(int argc, char *argv[])
{
    int v1; char c1;
    scanf("%c", &v1); // error
    scanf("%d", v1); // error
    scanf("%d %c", v1); // error
    scanf("%d %c", c1); // error
}
```

Example - 04



- `iostream` is part of the C++ standard library.
- It provides stream classes for performing input and output operations using an object-oriented approach.
- Frequently used streams
- `cin`, `cout`, `cerr`, `clog`
- These are defined in namespace `std`

```
#include<iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int v1;
    cin >> v1;
    cout << v1;
}
```

Compiling

```
$ g++ hello.cc -g
```

Executing

```
$ ./a.out
```




Example - 05

- `iostream` is part of the C++ standard library.
- It provides stream classes for performing input and output operations using an object-oriented approach.
- Frequently used streams
- `cin`, `cout`, `cerr`, `clog`
- These are defined in namespace `std`

```
#include<iostream>
// without using namespace

int main(int argc, char *argv[])
{
    int v1;
    std::cin >> v1;
    std::cout << v1;
}
```

Example - 06



- Without using namespace std and
- Without std::cin or
- std::cout
- Statements results in error

```
#include<iostream>
// without using namespace

int main(int argc, char *argv[])
{
    int v1;
    cin >> v1;
    cout << v1;
}
```

Example - 07



- The operator `<<` is bitwise left shift
- The operator `>>` is bitwise right shift
- **`1 << 3 = 8`**

```
#include<iostream>
```

```
int main(int argc, char *argv[])  
{  
    int v1 = 1 << 3;  
    printf("%d\n", v1)  
}
```

0	0	0	0	0	0	0	1
128	64	32	16	8	4	2	1

`1 << 3`

0	0	0	0	1	0	0	0
128	64	32	16	8	4	2	1



Example - 08

- The bitwise operators <<, >> when used with such as cin >> v1
- Or when used with cout << v1 perform the I/O operations
- They provides a more intuitive and readable syntax compared to stdio.h

```
cout << "Value of v1 is" << v1 << endl;
```

```
printf("Value of v1 is %d\n", v1);
```

Example - 09



- Reading inputs from a file
- Assume that you have a data file containing three lines, one integer per line
- File name is "input.txt"
- 10
- 20
- 30

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/01$ g++ p6.cc
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/01$ cat p6.cc
#include<iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int v1, v2, v3;
    cin >> v1 >> v2 >> v3;
    cout << v1 << endl << v2 << endl << v3 << endl;
}
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/01$ cat input.txt
10
20
30
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/01$ g++ p6.cc
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/01$ ./a.out < input.txt
10
20
30
```

```
.a.out < input.txt
```

Indirection operator



Data Types



Overview



- **Basic data types**
- **Modifying basic data types**
- Identifier names
- Variables
- Variable initialization
- Operators
- Expressions
- Basic data types - char, int, float, double, void, bool
- Modifiers: signed, unsigned, long, short

Overview



Type	Typical Size in Bits	Minimal Range
char	8	−127 to 127
unsigned char	8	0 to 255
signed char	8	−127 to 127
int	16 or 32	−32,767 to 32,767
unsigned int	16 or 32	0 to 65,535
signed int	16 or 32	same as int
short int	16	−32,767 to 32,767
unsigned short int	16	0 to 65,535
signed short int	16	same as short int
long int	32	−2,147,483,647 to 2,147,483,647
signed long int	32	same as long int
unsigned long int	32	0 to 4,294,967,295
float	32	Six digits of precision
double	64	Ten digits of precision
long double	80	Ten digits of precision



Identifiers & Variables



Overview



- Basic data types
- Modifying basic data types
- **Identifier names**
- Variables
- Variable initialization
- Operators
- Expressions
- The names of variables, functions, labels, and user-defined objects are called identifiers.
- Name can be one or more characters long
- First character must be from the set: {a, ..., z, A, ..., Z, _}
- Cannot have spaces or special characters like \$, !, #, %, :, ::, -, -> etc.

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- **Variables**
- Variable initialization
- Operators
- Expressions

- A variable is a named location in memory that is used to hold a value.

```
type variable_list;  
  
int i, j, k, l;  
  
short int si;  
  
unsigned int ui;  
  
double d;
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- **Variables**
- Variable initialization
- Operators
- Expressions
- Global variables are declared outside of all functions and are accessible from any part of the code after their declaration

```
#include<iostream>
using namespace std;

int j = 10; // global variable

int main(int argc, char *argv[])
{
    cout << j;
}
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- **Variables**
- Variable initialization
- Operators
- Expressions

- **Local variables are** declared inside a function and can only be accessed within that function

```
#include<iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int j = 10; // local variable
    cout << j;
}
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- **Variables**
- Variable initialization
- Operators
- Expressions

- **Static variables:** retain their value between function calls. They are initialized only once.

```
#include<iostream>
using namespace std;
void example()
{
    static int v1 = 0;
    v1 = v1 + 1;
    cout << v1;
}

int main(int argc, char *argv[])
{
    example();
    example();
    example();
}
```



Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- **Variable initialization**
- Operators
- Expressions

- Assigning value to variable.

```
int i1 = 10; // assign a value

char c1 = 'A'; // assign a value

// assign return value from a function
int i2 = example(p1);

i2 = i1 * 10 + example(p1); //expression
```



Operators



Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- Arithmetic operators

`+, -, *, /, %, ++, --`

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- Relational operators

`==, !=, <, <=, >, >=`

Overview



- Logical operators
- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

```
&& (and)  
|| (or)  
! (not)
```

Overview



- Bitwise operators
- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

```
& (bitwise and)
| (bitwise or)
^ (bitwise xor)
~ (bitwise not)
<< left shift
>> right shift
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- Assignment operators

```
= (assign)
+= (add and assign)
-= (subtract and assign)
*= (multiply and assign)
/= (divided by and assign)
%= (modulus and assign)
<<= (left shift and assign)
>>= (right shift and assign)
&= (bitwise and, assign)
|= (bitwise or and assign)
^= (bitwise xor and assign)
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- **Conditional operator**

```
int a = 10, b = 20;
```

```
int max = (a > b) ? a : b;
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- **Increment/decrement operator**

```
int a = 10, b = 20;  
  
a++;  
b--;
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- scope resolution operator

```
std::cout << variable;
```


Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- **Member access operators**

. (DOT) Used to access members of a structure or class.

-> (arrow) Used to access members of a structure or class through a pointer.

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

- **Pointer operators**

* Used to access the value at the address a pointer is pointing to

& Used to get the address of a variable

```
#include<iostream>
use namespace std;

int main(int argc, char *argv[])
{
    int source = 10;
    int *m;
    m = &source;
    int target = *m;
}
```

Overview



- new, delete operators
- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- **Operators**
- Expressions

Discussed shortly



Expressions



Overview



- Arithmetic expressions
- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- Expressions

```
#include <iostream>

int main(int argc, char *argv[])
{
    int a = 10;
    int b = 5;

    int sum = a + b;
    int difference = a - b;
    int product = a * b;
    int quotient = a / b;
    int remainder = a % b;

    std::cout << sum << std::endl;
    std::cout << difference << std::endl;
    std::cout << product << std::endl;
    std::cout << quotient << std::endl;
    std::cout << remainder << std::endl;

    return 0;
}
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- **Expressions**

- Relational expressions

```
#include <iostream>

int main(int argc, char *argv[])
{
    int a = 10;
    int b = 5;

    bool isEqual = (a == b);
    bool isNotEqual = (a != b);
    bool isGreater = (a > b);
    bool isLess = (a < b);
    bool isGeQ = (a >= b);
    bool isLeQ = (a <= b);

    std::cout << "Is =: " << isEqual << std::endl;
    std::cout << "Is !=: " << isNotEqual << std::endl;
    std::cout << "Is >: " << isGreater << std::endl;
    std::cout << "Is <: " << isLess << std::endl;
    std::cout << "Is >=: " << isGeQ << std::endl;
    std::cout << "Is <=: " << isLeQ << std::endl;

    return 0;
}
```

Overview



- Logical expressions
- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- Expressions

```
#include <iostream>

int main(int argc, char *argv[])
{
    bool a = true;
    bool b = false;

    bool logicalAnd = a && b;
    bool logicalOr = a || b;
    bool logicalNotA = !a;

    std::cout << "AND: " << logicalAnd << std::endl;
    std::cout << "OR: " << logicalOr << std::endl;
    std::cout << "NOTA:" << logicalNotA << std::endl;

    return 0;
}
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- **Expressions**

- Bitwise expressions

```
#include <iostream>

int main(int argc, char *argv[])
{
    int a = 6;    // 0110
    int b = 3;    // 0011

    int bitwiseAnd = a & b;
    int bitwiseOr = a | b;
    int bitwiseXor = a ^ b;
    int bitwiseNot = ~a;
    int leftShift = a << 1;
    int rightShift = a >> 1;

    std::cout << "AND: " << bitwiseAnd << std::endl;
    std::cout << "OR: " << bitwiseOr << std::endl;
    std::cout << "XOR: " << bitwiseXor << std::endl;
    std::cout << "NOT: " << bitwiseNot << std::endl;
    std::cout << "Left: " << leftShift << std::endl;
    std::cout << "Right: " << rightShift << std::endl;

    return 0;
}
```


Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- **Expressions**

- **Assignment expressions**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int a = 10;
    int b = 5;

    a += b;
    std::cout << "a += b: " << a << std::endl;

    a -= b;
    std::cout << "a -= b: " << a << std::endl;

    a *= b;
    std::cout << "a *= b: " << a << std::endl;

    a /= b;
    std::cout << "a /= b: " << a << std::endl;

    a %= b;
    std::cout << "a %= b: " << a << std::endl;
    return 0;
}
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- **Expressions**

- Conditional expressions

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int a = 10;
    int b = 20;

    int max = (a > b) ? a : b;

    std::cout << max << std::endl;

    return 0;
}
```

Overview



- Basic data types
- Modifying basic data types
- Identifier names
- Variables
- Variable initialization
- Operators
- **Expressions**

- Scope resolution expressions

```
#include <iostream>
using namespace std;

int a = 5;

int main(int argc, char *argv[])
{
    int a = 10;

    cout << "Local a: " << a << endl;
    cout << "Global a: " << ::a << endl;

    return 0;
}
```



Conditional statements, loops



Overview



- **Conditional statements**
- Loop structures
- Jump statements
- Block statements

- **If statement**

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int number = 10;

    if (number > 5)
    {
        cout << "number > 5." << endl;
    }
    else
    {
        cout << "number <= 5." << endl;
    }

    return 0;
}
```

Overview

- **Conditional statements**
- Loop structures
- Jump statements
- Block statements

- **Nested if statements**

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int age = 20;
    bool hasLicense = true;
    if (age >= 18)
    {
        if (hasLicense)
        {
            cout << "You are eligible to drive." << endl;
        }
        else
        {
            cout << "You need a driving license" << endl;
        }
    }
    else
    {
        if (age >= 16)
        {
            cout << "eligible for a learner's permit." << endl;
        }
        else
        {
            std::cout << "Too young to drive." << endl;
        }
    }

    return 0;
}
```



Overview

- **Conditional statements**
- Loop structures
- Jump statements
- Block statements

- **Switch statement**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int day = 3;
    switch (day) {
        case 1:
            std::cout << "Monday" << std::endl;
            break;
        case 2:
            std::cout << "Tuesday" << std::endl;
            break;
        case 3:
            std::cout << "Wednesday" << std::endl;
            break;
        case 4:
            std::cout << "Thursday" << std::endl;
            break;
        case 5:
            std::cout << "Friday" << std::endl;
            break;
        case 6:
            std::cout << "Saturday" << std::endl;
            break;
        case 7:
            std::cout << "Sunday" << std::endl;
            break;
        default:
            std::cout << "Invalid day" << std::endl;
            break;
    }
    return 0;
}
```



Overview

- Conditional statements
- **Loop structures**
- Jump statements
- Block statements

- For loop

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    for (int i = 0; i < 5; i++)
    {
        cout << "Iteration " << i << endl;
    }

    return 0;
}
```



Overview

- Conditional statements
- **Loop structures**
- Jump statements
- Block statements

- While loop

```
#include <iostream>
using namespace std;

int main()
{
    int i = 0;
    while (i < 5)
    {
        cout << "Iteration " << i << endl;
        i++;
    }

    return 0;
}
```



Overview

- Conditional statements
- **Loop structures**
- Jump statements
- Block statements

- **Do-While loop**

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int i = 0;
    do
    {
        cout << "Iteration " << i << endl;
        i++;
    } while (i < 5);

    return 0;
}
```





Jump statements



Overview

- Conditional statements
- Loop structures
- **Jump statements**
- Block statements

- Jump statements

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    for (int i = 0; i < 10; i++)
    {
        if (i == 5)
        {
            break; // jump statement
        }
        cout << "Iteration " << i << endl;
    }

    return 0;
}
```



Overview

- Conditional statements
- Loop structures
- **Jump statements**
- Block statements

- Jump statements

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    for (int i = 0; i < 10; i++)
    {
        if (i == 5)
        {
            continue; // jump statement
        }
        cout << "Iteration " << i << endl;
    }

    return 0;
}
```





Block structure statements



Overview

- Conditional statements
- Loop structures
- Jump statements
- **Block statements**

- **Block statements**

```
#include <iostream>
Using namespace std;

int main(int argc, char *argv[])
{
    int number = 10;

    // new block
    {
        int iNumber = 5;
        cout << "inner: " << iNumber << endl;
    }

    cout << "Outer: " << number << endl;

    return 0;
}
```





Arrays



Overview

- **Static array declaration**
- **Declare & Initialize**
- **Partial initialize**
- **Default initialization**
- **Dynamic array declaration**

- **Static one dimensional arrays**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int arr[5];
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 3;
    arr[3] = 4;
    arr[4] = 5;

    for (int i = 0; i < 5; i++)
    {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```



Overview

- Static array declaration
- **Declare & Initialize**
- Partial initialize
- Default initialization
- Dynamic array declaration

- **Declare & initialize**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int arr[5] = {1, 2, 3, 4, 5};

    for (int i = 0; i < 5; i++)
    {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```



Overview

- Static array declaration
- Declare & Initialize
- **Partial initialize**
- Default initialization
- Dynamic array declaration

- Partial initialization

```
#include <iostream>

int main(int argc, char *argv[])
{
    int arr[5] = {1, 2}; // rest 0

    for (int i = 0; i < 5; i++)
    {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```



Overview

- Static array declaration
- Declare & Initialize
- Partial initialize
- **Default initialization**
- Dynamic array declaration

- **Default initialization**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int arr[5] = {}; // all elements 0

    for (int i = 0; i < 5; i++)
    {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```



Overview

- Static array declaration
- Declare & Initialize
- Partial initialize
- Default initialization
- **Dynamic array declaration**

- **Dynamic array declaration**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int *arr = NULL;
    arr = (int *) malloc(5 * sizeof(int) );

    for (int i = 0; i < 5; i++)
    {
        arr[i] = 10 + i;
    }

    for (int i = 0; i < 5; i++)
    {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```



Overview

- Static array declaration
- Declare & Initialize
- Partial initialize
- Default initialization
- **Dynamic array declaration**

- **Dynamic array declaration**

```
#include <iostream>

int main(int argc, char *argv[])
{
    int *arr = new int[5];
    for (int i = 0; i < 5; i++)
    {
        arr[i] = 10 + i;
    }

    for (int i = 0; i < 5; i++)
    {
        std::cout << arr[i] << " ";
    }

    return 0;
}
```



Overview



new operator

- **new** is type-safe and returns a pointer of the appropriate type, eliminating the need for explicit casting.

```
int *arr = new int[5];
```

malloc function

- **malloc** returns a void* that must be cast to the desired type.

```
arr = (int *) malloc(5 * sizeof(int));
```

Overview



new operator

- new can initialize the allocated memory, including calling constructors for

```
#include<iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int *arr = new int[5];

    for( int i = 0; i < 5; i++ )
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

malloc function

- malloc does not initialize memory

```
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char *argv[])
{
    int *arr = NULL;
    arr = (int *) malloc(5 * sizeof(int));

    int i = 0;
    for( i = 0; i < 5; i++ )
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);

    return 0;
}
```


Overview



new operator

- Memory allocated with new must be deallocated with delete or delete[],
- delete calls destructors for class objects.

```
int *arr = new int[5];  
  
delete[] arr;
```

malloc function

- Memory allocated with malloc must be deallocated with free
- free does not call destructors.

```
arr = (int *) malloc(5 * sizeof(int));  
  
free(arr);
```

Overview



- Other methods of array declaration and initialization
- Using STL class array
- Using vector
- We will discuss these two methods after the theme "class" and "template"



Two dimensional Arrays



Overview



- Static two dimensional arrays

- **Static 2D array declaration**
- **Declare & Initialize**
- **Partial initialize**
- **Dynamic array declaration**

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int arr[3][3];
    arr[0][0] = 1; arr[0][1] = 2; arr[0][2] = 3;
    arr[1][0] = 4; arr[1][1] = 5; arr[1][2] = 6;
    arr[2][0] = 7; arr[2][1] = 8; arr[2][2] = 9;

    for (int i = 0; i < 3; ++i)
    {
        for (int j = 0; j < 3; ++j)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Overview

- Static 2D array declaration
- **Declare & Initialize**
- Partial initialize
- Dynamic array declaration

- Static 2D arrays – declare & initialize



```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int arr[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    for (int i = 0; i < 3; ++i)
    {
        for (int j = 0; j < 3; ++j)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Overview

- Static 2D array declaration
- Declare & Initialize
- **Partial initialize**
- Dynamic array declaration

- Partial initialization of 2D arrays



```
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int arr[3][3] = { {1, 2}, {3, 4} };

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Overview

- Dynamic 2D arrays



- Static 2D array declaration
- Declare & Initialize
- Partial initialize
- **Dynamic array declaration**

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int rows = 3;
    int cols = 3;
    int** arr = new int*[rows];
    for (int i = 0; i < rows; ++i)
    {
        arr[i] = new int[cols];
    }

    for (int i = 0; i < rows; i++)
    {
        delete[] arr[i];
    }
    delete[] arr;

    return 0;
}
```



Pointers



Pointers



- Declaration & De-referencing
- Pointer Assignment
- Pointer Arithmetic
- Pointers & Arrays
- Pointers to pointers

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int var = 10;
    int* ptr = &var;

    cout << "Value of var: " << var << endl;
    cout << "Address of var: " << &var << endl;
    cout << "Value stored in ptr: " << ptr << endl;
    cout << "Value pointed to by ptr: " << *ptr << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024$ g++ p1.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024$ ./a.out
Value of var: 10
Address of var: 0x7ffd1e2d8c9c
Value stored in ptr: 0x7ffd1e2d8c9c
Value pointed to by ptr: 10
```

Pointers



- Declaration & De-referencing
- **Pointer Assignment**
- Pointer Arithmetic
- Pointers & Arrays
- Pointers to pointers

```
#include<iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int x = 10;
    int *p1, *p2;

    p1 = &x;
    p2 = p1;

    cout << "Address of x: " << &x << endl;
    cout << "Value stored in p1: " << p1 << endl;
    cout << "Value stored in p2: " << p2 << endl;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024$ g++ p5.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024$ ./a.out
Address of x: 0x7fff90f48414
Value stored in p1: 0x7fff90f48414
Value stored in p2: 0x7fff90f48414
```

Pointers



- Declaration & De-referencing
- Pointer Assignment
- **Pointer Arithmetic**
- Pointers & Arrays
- Pointers to pointers

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int arr[3] = {10, 20, 30};
    int* ptr = arr;

    cout << "arr[0]: " << *ptr << endl;

    ptr++;
    cout << "arr[1]: " << *ptr << endl;

    ptr++;
    cout << "arr[2]: " << *ptr << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024$ g++ p2.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024$ ./a.out
arr[0]: 10
arr[1]: 20
arr[2]: 30
saradhi@saradhi:~/courses/cs204/july-dec-2024$
```

Pointers



- Declaration & De-referencing
- Pointer Assignment
- **Pointer Arithmetic**
- Pointers & Arrays
- Pointers to pointers

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int arr[3] = {10, 20, 30};
    int* ptr = &arr[1];

    cout << "arr[1]: " << *ptr << endl;

    ptr--;
    cout << "arr[0]: " << *ptr << endl;

    ptr++;
    ptr++;
    cout << "arr[2]: " << *ptr << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024$ g++ p2-2.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024$ ./a.out
arr[1]: 20
arr[0]: 10
arr[2]: 30
saradhi@saradhi:~/courses/cs204/july-dec-2024$
```



Pointers

- Declaration & De-referencing
- Pointer Assignment
- Pointer Arithmetic
- **Pointers & Arrays**
- Pointers to pointers

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int i = 0;
    int arr[3] = {10, 20, 30};
    int* ptr = arr;

    for (i = 0; i < 3; i++)
    {
        cout << "Element " << i << " of array: " << *(ptr + i) << endl;
    }

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024$ g++ p3.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024$ ./a.out
Element 0 of array: 10
Element 1 of array: 20
Element 2 of array: 30
```

Pointers

- Declaration & De-referencing
- Pointer Assignment
- Pointer Arithmetic
- Pointers & Arrays
- Pointers to pointers



```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    int var = 10;
    int* ptr = &var;
    int** pptr = &ptr;

    cout << "Value of var: " << var << endl << endl;
    cout << "Address of var: " << &var << endl;

    cout << "Value stored in ptr: " << ptr << endl;
    cout << "Value pointed to by ptr: " << *ptr << endl << endl;

    cout << "Address of ptr : " << &ptr << endl;
    cout << "Value stored in pptr: " << pptr << endl;
    cout << "Value pointed to by pptr: " << **pptr << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024$ g++ p4.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024$ ./a.out
Value of var: 10

Address of var: 0x7ffe57807764
Value stored in ptr: 0x7ffe57807764
Value pointed to by ptr: 10

Address of ptr : 0x7ffe57807768
Value stored in pptr: 0x7ffe57807768
Value pointed to by pptr: 10
```

Lab #01



- **First lab will be based on writing a basic C++ program**
- **Theme will be to demonstrate the use of conditional statements, loop structures, arrays to perform a given task.**
- **The programs require the use of built-in data types**



Questions?





Thank You

